Estudio de ASProtect v1.23 RC4 - v1.3.08.24

Programa:	TagRename v2.1.7.4
Protección:	ASProtect v1.23 RC4 -1.3.08.24 + Trial 30 días
Objetivos:	Desempakar y evitar el trial
URL descarga:	http://DondeSea.com
Dificultad:	Facilón
Herramientas:	OllyDbg v1.10 + PEiD v0.94 + ImpREC v1.6 + ConTEXT v0.98 + Hex WorkShop 4.23 + PEditor v1.7 + LordPE Deluxe
Cracker:	SyXe'05 .:[CracksLatinoS]:. Fecha: 17-06-06

Un saludo a todo CracksLatinoS!

Volvemos a vernos las caras otra vez, jeje.. hace días que no escribo nada y tenía ganas de hacer algo, lo que sea! Y mirando entre mis carpetas de time-trials (tengo más de 3000, jejeje.. y no es broma) encontré un programita que todavía no sé bien lo que hace pero creo que servirá para el tutorial, aún no lo he crackeado y no sé si lo podré hacer, pero me aventuro a comenzar el tute, si lo acabo pues perfecto y sino pues nada irá a la carpeta de tutes empezados sin terminar ;-(

Esperemos que no sea ese su futuro ;-) comenzamos como siempre instalando la app, abrimos PEiD 0.94 y echamos un vistazo:

ASProtect 1.23 RC4 - 1.3.08.24 -> Alexey Solodovnikov

Vale un ASProtect de los medianos, ni muy nuevo ni muy viejo digamos, vamos a por él, cerramos PEiD y nos abrimos el nuevo y flamante OllyIce cortesía de hacnho y que pueden descargar de la siguiente página Web:

http://www.tinicat.de/hacnho

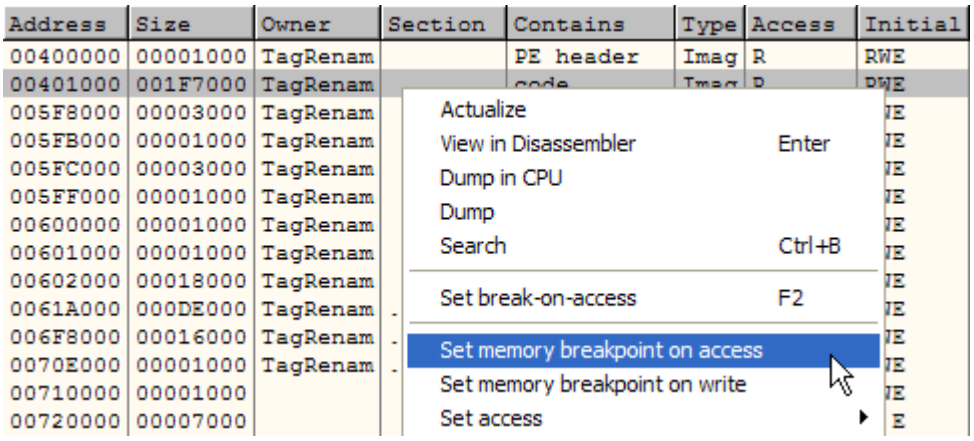
Y allí mismo pone esto:

"And now, my last gift for you: OllyICE FULL All In One: Cool style-All Plugins-Disasm very fast-Anti All Anti DebugMethods-AllScripts-LIB Included!"

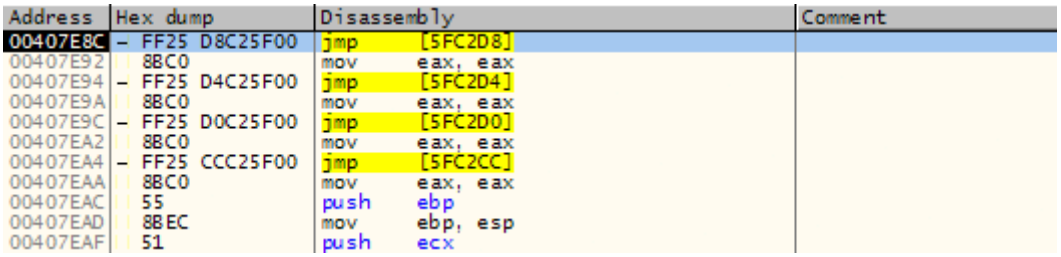
Según dice un estilo fresco, con todos los plugins, rápido desensamblado y oculta todos los métodos antidebug ¿? Parece una joya, lo probaremos a ver qué tal va. Cargamos el ejecutable empakado "TagRename.exe" y quedamos detenidos en su EntryPoint, desde aquí vamos superando excepciones hasta llegar a la última, esta en mi caso:

```
Address   Hex dump          Disassembly                    Comment
014D39EC  3100              xor     [eax],eax              // ultima excepcion
014D39EE  64:8F05 000000    pop     dword ptr fs:[0]
014D39F5  58                pop     eax
014D39F6  833D B07E4D01     cmp     dword ptr [14D7EB0],0
014D39FD  74 14             je      short 014D3A13
014D39FF  6A 0C             push    0C
014D3A01  B9 B07E4D01       mov     ecx,14D7EB0
```

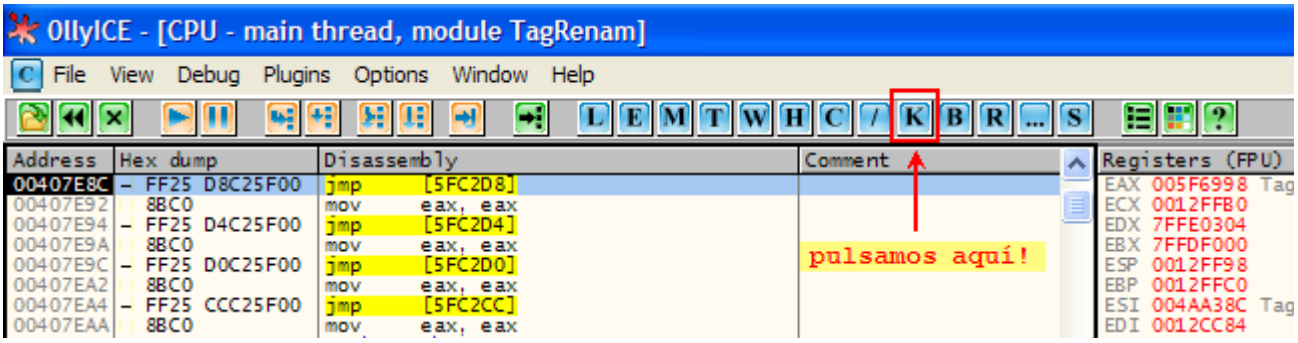
No la superaremos sino que iremos a la ventana de memoria con ALT+M y pondremos un BPM en toda la primera sección (la de código):



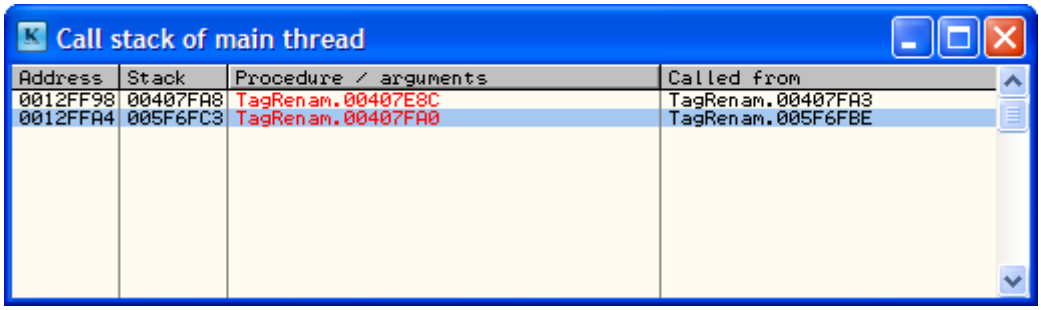
Ahora sí superamos la excepción con SHIFT+F9 y este es el resultado:



El OEP? Por su puesto que no, queda claro que tenemos Stolen Bytes aquí, caímos justo en uno de los saltos a la IAT (normalmente apunta a GetModuleHandleA) antes de llegar aquí hemos debido de entrar en algún CALL, lo miraremos desde el Call Stack (ALT+K):



Al pulsar sobre la tecla de la "K" se abre la ventanita correspondiente donde veremos las direcciones de los CALL que han transcurrido hasta llegar aquí:



El seleccionado es el que más nos interesa de los dos ya que el de arriba nos deja donde estamos ahora (en el JMP), apretamos Enter y caemos en la parte que corresponde a esta llamada, nos deja aquí:

```
Address   Hex dump     Disassembly                          Comment
00407F9D  8D40 00      lea     eax, [eax]
00407FA0  50           push    eax
00407FA1  6A 00        push    0
00407FA3  E8 E4FEFFFF  call    00407E8C
00407FA8  BA 08815F00  mov     edx, 005F8108
00407FAD  52           push    edx
00407FAE  8905 DCB45F00 mov    [5FB4DC], eax
00407FB4  8942 04      mov     [edx+4], eax
00407FB7  C742 08 000000( mov  dword ptr [edx+8], 0
00407FBE  C742 0C 000000( mov  dword ptr [edx+C], 0
00407FC5  E8 82FFFFFF  call    00407F4C
00407FCA  5A           pop     edx
00407FCB  58           pop     eax
00407FCC  E8 6BC3FFFF  call    0040433C
00407FD1  C3           retn
00407FD2  8BC0         mov     eax, eax
00407FD4  55           push    ebp
00407FD5  8BEC         mov     ebp, esp
```

No vemos los Stolen Bytes por ningún lado ¿? Encima del "lea eax, [eax]" que por cierto es una instrucción inválida no vemos los típicos ceros que los representan, para contrarrestar esto lo que haremos será descubrirlos de otra manera: ahora estamos en una rutina del programa, lo sabemos porque venimos del salto a la IAT en que estábamos antes y porque al final vemos un RETN, pues allí ponemos un Bp como vemos en la imagen anterior, pulsamos F9 y en un instante Olly se detiene allí:

```
Address   Hex dump     Disassembly                          Comment
00407FCB  58           pop     eax
00407FCC  E8 6BC3FFFF  call    0040433C
00407FD1  C3           retn
00407FD2  8BC0         mov     eax, eax
```

Pasamos el RETN apretando F8 y caemos en esta parte:

```
Address   Hex dump     Disassembly                          Comment
005F6FBF  DD           ???                                  Unknown command
005F6FC0  0FE1FF       psraw   mm7, mm7
005F6FC3  68 98705F00  push    005F7098                     ASCII "TAG_AND_RENAME
005F6FC8  6A FF        push    -1
005F6FCA  6A 00        push    0
005F6FCC  E8 CB10E1FF  call    0040809C
005F6FD1  A3 74BF5F00  mov     [5FBF74], eax
005F6FD6  33C9         xor     ecx, ecx
005F6FD8  55           push    ebp
005F6FD9  68 86705F00  push    005F7086
005F6FDE  64:FF31      push    dword ptr fs:[ecx]
005F6FE1  64:8921      mov     fs:[ecx], esp
005F6FE4  BA 0D000000  mov     edx, 0D
005F6FE9  B8 04A75F00  mov     eax, 005FA704
005F6FEE  8028 02      sub     byte ptr [eax], 2
005F6FF1  40           inc     eax
005F6FF2  4A           dec     edx
005F6FF3  ^ 75 F9      jnz     short 005F6FEE
005F6FF5  33C0         xor     eax, eax
005F6FF7  55           push    ebp
005F6FF8  68 47705F00  push    005F7047
```

Aparentemente nada extraño salvo ese **psraw mm7, mm7** totalmente ilógico y la instrucción ??? que nos indica que algo no va bien, hacemos un scroll hacia arriba con el ratón y qué encontramos? nuestros stolen bytes, jejeje…

```
Address   Hex dump     Disassembly                          Comment
005F6FAF  0000         add     [eax],al
005F6FB1  0000         add     [eax],al
005F6FB3  0000         add     [eax],al
005F6FB5  0000         add     [eax],al
005F6FB7  0000         add     [eax],al
005F6FB9  0000         add     [eax],al
005F6FBB  0000         add     [eax],al
005F6FBD  00E8         add     al,ch
005F6FBF  DD           ???                                  Unknown command
005F6FC0  0FE1FF       psraw   mm7,mm7
005F6FC3  68 98705F00  push    005F7098                     ASCII "TAG_AND_RENAME"
005F6FC8  6A FF        push    -1
005F6FCA  6A 00        push    0
005F6FCC  E8 CB10E1FF  call    0040809C
005F6FD1  A3 74BF5F00  mov     [5FBF74],eax
```

Parece que faltan unas cuantas instrucciones, de momento nuestro falso OEP estará en :5F6FBE luego ya veremos donde queda el verdadero.

El siguiente punto será reparar la IAT, podríamos buscar los stolen bytes ahora pero a mi personalmente me gusta más hacerlo al final, volvemos al JMP donde estábamos antes, lo seleccionamos y hacemos un FOLLOW IN DUMP -> MEMORY ADDRESS para caer en medio de la IAT, desde allí subimos con los cursores hasta encontrar su inicio:

```
Address   Hex dump                                          ASCII
005FC1AC  F5 62 CF 88 3E B8 BB 6D C8 B1 04 F0 3D 49 A3 2F  õbÏˆ>¸»mÈ±·ð=I£/
005FC1BC  E7 43 D5 F8 18 22 8A AB 8C 16 BC 2B 92 8A EB D0  çCÕø↑"Šœ←¼+'ŠëÐ
005FC1CC  4C 03 54 01 6C 04 54 01 80 04 54 01 8C 04 54 01  L♥T┘l♦T┘€♦T┘Œ♦T┘
005FC1DC  F8 02 54 01 98 04 54 01 B0 04 54 01 CC 04 54 01  ø☻T┘˜♦T┘°♦T┘Ì♦T┘
005FC1EC  E0 04 54 01 F4 04 54 01 00 05 54 01 0C 05 54 01  à♦T┘ô♦T┘.♣T┘.♣T┘
005FC1FC  24 05 54 01 34 05 54 01 48 05 54 01 5C 05 54 01  $♣T┘4♣T┘H♣T┘\♣T┘
005FC20C  70 05 54 01 84 05 54 01 94 05 54 01 A4 05 54 01  p♣T┘„♣T┘"♣T┘¤♣T┘
005FC21C  A4 17 4D 01 64 1C 4D 01 B8 05 54 01 D0 05 54 01  ¤↨M┘d∟M┘¸♣T┘Ð♣T┘
```

Aquí lo encontramos, nos apuntamos el dato:

Inicio_IAT: 5FC1CC

Hacemos lo mismo con el final, aquí lo vemos:

```
Address   Hex dump                                          ASCII
005FCA1C  62 8B B0 76 B8 9D BF 93 E7 43 1B 76 23 F0 B9 68  b‹°v¸¢¿"çC←v#ð'h
005FCA2C  7C D5 3C 77 31 27 40 77 56 B0 3F 77 1D C5 3C 77  |Õ<w1'@wV°?w↔Å<w
005FCA3C  C2 35 4B 77 A1 60 2F AB 05 31 37 76 CF 90 36 76  Â5Kw¡`/«♣17vÏ6v
005FCA4C  90 23 36 76 88 25 7B 36 3C DD FC 00 22 82 8E E6  ¤#6vˆ%{6<Ýü.",Žæ
005FCA5C  2B 3B E7 77 90 32 E9 77 DF AA 55 2A CF 90 36 76  +;çw¤2éwßªU*Ï¤6v
005FCA6C  90 23 36 76 05 F3 75 0A D7 4D 3E 77 68 16 3E 77  ¤#6v♣óu◙×M>wh▬>w
005FCA7C  EB CB CD B3 15 98 3E 56 5B 09 A6 78 35 8E 88 2D  ëËÍ³§↑˜>V[○¦x5Žˆ-
005FCA8C  55 2E 72 FA 02 20 9A 6E 11 C7 88 C2 5E C8 16 63  U.rú☻ šn◄Çˆ^È▬c
```

lo anotamos también que luego nos hará falta:

Final_IAT: 5FCA7C

Como siempre le damos un pequeño margen sino a veces el IMPORT RECONSTRUCTOR nos deja la última entrada fuera, así lo evitamos. Ahora, calcular su longitud es simplemente restando ambos valores, queda esto:

Longitud_IAT: 8B0

Estos tres datos los guardamos para luego, lo que vamos a hacer ahora es localizar el salto mágico que evita que parte de la IAT sea redireccionada hacia el código del packer, esta tarea es sencilla: nos fijamos en una entrada MALA, esta por ejemplo:

```
Address   Hex dump                                          ASCII
005FC1CC  4C 03 54 01 6C 04 54 01 80 04 54 01 8C 04 54 01  L♥T┘l♦T┘€♦T┘Œ♦T┘
005FC1DC  F8 02 54 01 98 04 54 01 B0 04 54 01 CC 04 54 01  ø☻T┘˜♦T┘°♦T┘Ì♦T┘
005FC1EC  E0 04 54 01 F4 04 54 01 00 05 54 01 0C 05 54 01  à♦T┘ô♦T┘.♣T┘.♣T┘
```

Nos quedamos con su valor final 0154034C, reiniciamos Olly, le ponemos un BP - MEMORY, ON WRITE y pulsamos F9 y para cuando escribir algo allí:

```
Address   Hex dump    Disassembly                  Comment
014D32B0    8B45 0C     mov     eax,[ebp+C]
014D32B3    50          push    eax
014D32B4    E8 47FCFFFF call    014D2F00
014D32B9    E8 7EFEFFFF call    014D313C  <------  cuidado con este CALL
014D32BE    8B17        mov     edx,[edi]
014D32C0    8902        mov     [edx],eax
014D32C2  v EB 7E       jmp     short 014D3342
```

Olly paró en :014D32C0 si miramos los registros ahora vemos esto:

```
Registers (FPU)
EAX 0154034C
ECX 00000000
EDX 005FC1CC TagRenam.005FC1CC
EBX 00000013
ESP 0012FE1C
EBP 0012FF2C
ESI 0012FE2B ASCII "GetCurrentThreadId"
EDI 0012FF80
```

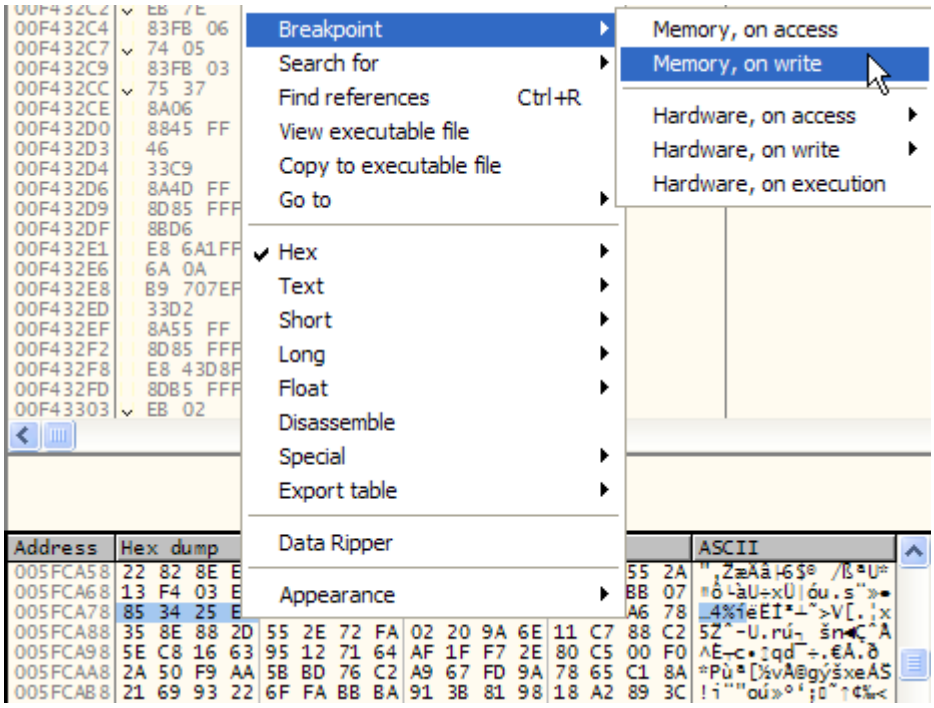
claramente vemos que está rellenando la IAT en estos momentos, el valor de EAX será el próximo a guardar y como vemos está redirigido (Api_encriptada), la culpa la tiene el CALL de :014D32B9 lo tenemos que saltear para que no se ejecute, bien mediante script o topeándolo, el tema es que no se ejecute ya que allí se hace el intercambio Api_buena por Api_encriptada.

Reiniciamos Olly y pulsamos F9, cuando pare en la primera excepción hacemos un Go al call mágico, le ponemos un BP y superamos la excepción con SHIFT+F9 y caemos sobre esta CALL [Mágica], la nopeamos:

```
Address   Hex dump    Disassembly          Comment
014D32B0  8B45 0C     mov     eax,[ebp+C]
014D32B3  50          push    eax
014D32B4  E8 47FCFFFF call    014D2F00
014D32B9  90          nop
014D32BA  90          nop
014D32BB  90          nop
014D32BC  90          nop
014D32BD  90          nop
014D32BE  8B17        mov     edx,[edi]
```

Así debe quedar, es posible que debamos desactivar las excepciones desde el cuadro OPTIONS - DEBUGGING OPTIONS - EXCEPTIONS [ALT+O] para que no se detenga e ir más rápido, sólo las dejaremos así hasta topear el call, después las dejamos tal cual estaban para luego. Hecho esto pondremos un BPM en la última entrada de la IAT para que se detenga al escribir allí y restaurar el CALL nopeado, pues muy posiblemente no nos deje llegar al OEP si no lo dejamos intacto, pues como vimos en otros tutes de este packer utiliza el CRC del mismo para desencriptar valores de retorno en ciertas CALLs, así que ojo con poner BPs que sucede lo mismo…

Nos vamos a la última entrada de la tabla y colocamos un BPM ON WRITE:



Y pulsamos F9, enseguida Olly para aquí:

```
Address   Hex dump    Disassembly          Comment
014D332A  8B45 0C     mov     eax,[ebp+C]
014D332D  50          push    eax
014D332E  E8 CDFBFFFF call    014D2F00
014D3333  8B17        mov     edx,[edi]
014D3335  8902        mov     [edx],eax    shell32.SHChangeNotifyRegister
014D3337  EB 09       jmp     short 014D3342
```

Esa es la última, ahora nos desplazamos a la posición del CALL mágico y le hacemos un UNDO SELECTION para que quede como está

```
Address   Hex dump     Disassembly                  Comment
014D32AF    56         push    esi
014D32B0    8B45 0C    mov     eax,[ebp+C]
014D32B3    50         push    eax
014D32B4    E8 47FCFFFF call    014D2F00
014D32B9    90         nop
014D32BA    90         nop
014D32BB    90         nop
014D32BC    90         nop
014D32BD    90         nop
014D32BE    8B17       mov     edx,[e
014D32C0    8902       mov     [edx],
014D32C2  v EB 7E      jmp     short
014D32C4    83FB 06    cmp     ebx,6
014D32C7  v 74 05      je      short
014D32C9    83FB 03    cmp     ebx,3
014D32CC  v 75 37      jnz     short
```

Backup
Copy
Binary
Undo selection Alt+BkSp
Assemble Space

Otra forma más sencilla sería en vez colocar del BPM en la última entrada [que a veces desconocemos] poner un BP con F2 en el POPAD que hay al final del loop donde rellena la tabla:

```
Address   Hex dump     Disassembly                  Comment
014D354D    5B         pop     ebx
014D354E  ^ EB CE      jmp     short 014D351E
014D3550    61         popad                        // Aqui el POPAD!!
014D3551    E8 3A000000 call   014D3590
014D3556    68 5F354D01 push   14D355F
014D355B    FF0424     inc     dword ptr [esp]
014D355E    C3         ret
```

Después haríamos igual el Undo Selection para restaurar el call NOPeado, activaríamos nuevamente las excepciones y llegaríamos al OEP con la tabla casi resuelta puesto que nos quedan algunos punteros por resolver todavía, que son:

Entrada	IAT rva:	Apunta a:	API correcta:
1.-	1FC21C	014D17A4	GetProcAddress
2.-	1FC220	014D1C64	GetModuleHandleA
3.-	1FC230	014D1CD8	GetCommandLineA
4.-	1FC2D8	014D1C64	GetModuleHandleA
5.-	1FC378	014D1CC8	LockResource
6.-	1FC3B8	014D1C8C	GetVersion
7.-	1FC3DC	014D17A4	GetProcAddress
8.-	1FC3E0	014D1C64	GetModuleHandleA
9.-	1FC414	014D1CC0	GetCurrentProcessId
12.-	1FC420	014D1CF0	FreeResource

Y dejamos todas las entradas reparadas:

Imported Functions Found

```
+ kernel32.dll FThunk:001FC1CC NbFunc:2E (decimal:46) valid:YES
+ user32.dll FThunk:001FC288 NbFunc:4 (decimal:4) valid:YES
+ advapi32.dll FThunk:001FC29C NbFunc:3 (decimal:3) valid:YES
+ oleaut32.dll FThunk:001FC2AC NbFunc:7 (decimal:7) valid:YES
+ kernel32.dll FThunk:001FC2CC NbFunc:5 (decimal:5) valid:YES
+ advapi32.dll FThunk:001FC2E4 NbFunc:C (decimal:12) valid:YES
+ kernel32.dll FThunk:001FC318 NbFunc:5D (decimal:93) valid:YES
+ version.dll FThunk:001FC490 NbFunc:3 (decimal:3) valid:YES
+ gdi32.dll FThunk:001FC4A0 NbFunc:63 (decimal:99) valid:YES
```

De momento salvamos la tabla en SAVE TREE y la llamamos iat.txt, no nos debemos preocupar por el OEP ya que luego veremos cómo recuperar los stolen bytes y veremos la dirección exacta de éste, de momento en el campo OEP dejamos el 0x1000h que viene por defecto.

Y ahora sí vamos a por los stolen bytes, sabemos que hay varias posibilidades a partir de aquí, es posible que los encontremos sin dificultad y también podría ser que no los encontremos, en cualquier caso reconstruiremos el ejecutable, pues en el peor de los casos nos apoyaremos en la pila como nos enseñó **Juan José** en sus tutes de ASProtect [un saludo amigo ;)] o Ricardo en los suyos [otro saludo para ti xD]

Lo que haremos será llegar a la última excepción en nuestra máquina que ya conocemos, una vez allí mirar la pila y ver donde se encuentra el SEH [Manejador de excepciones], en mi caso veo esto:

Address	Value	Comment
0012FF64	0012FFE0	Pointer to next SEH record
0012FF68	014D39CF	SE handler
0012FF6C	014C0000	
0012FF70	014A0000	
0012FF74	014D4138	
0012FF78	0154AE18	
0012FF7C	015469B4	

pertenece al bloque [014C0000 - 014DE000]

Voy al MEMORY MAP [ALT+M] y coloco un BPM ON MEMORY ACCESS en dicho bloque de memoria donde se encuentra el SEH que manejará la excepción, de esta manera me aseguro de poder continuar el traceo antes de saltar al OEP, puesto el BPM aprieto SHIFT+F9 para pasar la excepción y caigo aquí:

Address	Hex dump	Disassembly		Comment
014D39CF	8B4424 0C	mov	eax,[esp+C]	// Olly para aqui
014D39D3	8380 B8000000	add	dword ptr [eax+B8],2	
014D39DA	C740 18 000000	mov	dword ptr [eax+18],0	
014D39E1	31C0	xor	eax,eax	
014D39E3	C3	ret		// Me lleva a NTDLL.DLL
014D39E4	31C0	xor	eax,eax	
014D39E6	64:FF30	push	dword ptr fs:[eax]	
014D39E9	64:8920	mov	fs:[eax],esp	
014D39EC	3100	xor	[eax],eax	
014D39EE	64:8F05 000000	pop	dword ptr fs:[0]	// luego para aqui..
014D39F5	58	pop	eax	
014D39F6	833D B07E4D01	cmp	dword ptr [14D7EB0],0	
014D39FD	74 14	je	short 014D3A13	
014D39FF	6A 0C	push	0C	
014D3A01	B9 B07E4D01	mov	ecx,14D7EB0	
014D3A06	8D45 F8	lea	eax,[ebp-8]	
014D3A09	BA 04000000	mov	edx,4	
014D3A0E	E8 2DD1FFFF	call	014D0B40	
014D3A13	FF75 FC	push	dword ptr [ebp-4]	
014D3A16	FF75 F8	push	dword ptr [ebp-8]	
014D3A19	8B45 F4	mov	eax,[ebp-C]	
014D3A1C	8338 00	cmp	dword ptr [eax],0	
014D3A1F	74 02	je	short 014D3A23	
014D3A21	FF30	push	dword ptr [eax]	
014D3A23	FF75 F0	push	dword ptr [ebp-10]	
014D3A26	FF75 EC	push	dword ptr [ebp-14]	
014D3A29	C3	ret		

Olly se detiene en :014D39CF, traceo hasta el RET que me deja en la librería de sistema NTDLL, como tengo puesto el BPM aprieto nuevamente F9 y me deja un poco más abajo [en :014D39EE] desde aquí traceo con F8 hasta el siguiente RET que me deja en otra sección del packer, justo aquí:

Address	Hex dump	Disassembly		Comment
015469B4	66:81D7 8387	adc	di,8783	// Aqui me deja..
015469B9	E8 05000000	call	015469C3	
015469BE	397E DF	cmp	[esi-21],edi	
015469C1	2C F5	sub	al,0F5	
015469C3	8BFE	mov	edi,esi	
015469C5	59	pop	ecx	
015469C6	66:8BD9	mov	bx,cx	
015469C9	81C1 84150000	add	ecx,1584	
015469CF	50	push	eax	
015469D0	E9 14000000	jmp	015469E9	

Lo siguiente que hago es tracear con mucho cuidado y siempre con F7 para no saltear ningún call puesto que al hacer esto es muy probable que aparezca el programa de golpe sin que Olly pare, luego hay que deshacer algún que otro bucle como este poniendo un BP al final del mismo y apretando F9:

Address	Hex dump	Disassembly		Comment
01546A71	5F	pop	edi	
01546A72	4E	dec	esi	
01546A73	0F85 11000000	jnz	01546A8A	
01546A79	B8 7D0B367B	mov	eax,7B360B7D	// BP aqui..
01546A7E	E9 17000000	jmp	01546A9A	
01546A83	C3	ret		
01546A84	40	inc	eax	
01546A85	79 BE	jns	short 01546A45	
01546A87	1F	pop	ds	Modification of segment
01546A88	6C	insb		I/O command
01546A89	35 8BD8E972	xor	eax,72E9D88B	
01546A8E	FFFF	???		Unknown command
01546A90	FFED	jmp	far ebp	Illegal use of register
01546A92	22B3 70E96E0F	and	dh,[ebx+F6EE970]	
01546A98	9C	pushfd		

Pongo un BP en :01546A79 y pulso F9 para caer allí, me doy cuenta que parte del código que hay debajo cambió así que estaba desempakando código del propio packer, continúo con F7 y llego a otro loop:

```
Address   Hex dump        Disassembly                Comment
01546AC8  0FB703         movzx   eax,word ptr [ebx]
01546ACB  03C6           add     eax,esi
01546ACD  83C3 02        add     ebx,2
01546AD0  8BD0           mov     edx,eax
01546AD2  8BC6           mov     eax,esi
01546AD4  E8 0C000000    call    01546AE5
01546AD9  66:833B 00     cmp     word ptr [ebx],0
01546ADD  75 E9          jnz     short 01546AC8
01546ADF  5E             pop     esi
01546AE0  5B             pop     ebx
01546AE1  5D             pop     ebp
01546AE2  C2 0800        ret     8
01546AE5  0102           add     [edx],eax
01546AE7  C3             ret
```

Aquí pulso CTRL+F9 para salir de éste y se detiene en un POPAD que hay un poco más debajo de este buclecillo:

```
Address   Hex dump       Disassembly                Comment
01546B3E  51             push    ecx
01546B3F  50             push    eax
01546B40  E8 76FFFFFF    call    01546ABB
01546B45  61             popad
01546B46  F3:            prefix rep:                Superfluous prefix
01546B47  EB 02          jmp     short 01546B4B
01546B49  CD 20          int     20
01546B4B  2E:EB 02       jmp     short 01546B50     Superfluous prefix
01546B4E  CD 20          int     20
01546B50  8D6424 D5      lea     esp,[esp-2B]
01546B54  2E:EB 02       jmp     short 01546B59     Superfluous prefix
01546B57  CD 20          int     20
```

A partir de aquí hay que prestar mucha atención ya que los stolen bytes están a punto de ejecutarse, si miramos la pila vemos que aún o se ejecutó nada:

```
Address   Value     Comment
0012FFC4  77E614C7  RETURN to kernel32.77E614C7
0012FFC8  77D121AE  user32.77D121AE
0012FFCC  0012CD34
0012FFD0  7FFDF000
0012FFD4  F708CCF0
0012FFD8  0012FFC8
0012FFDC  8053D14F
```

Pila limpita ;)

Ese valor 0x77E614C7 es el valor de retorno al Sistema Operativo cuando el termine la ejecución del programa donde se llamará a ExitThread, así que como digo todavía no se ejecutó ningún stolen byte, al momento llegamos a:

```
Address   Hex dump        Disassembly                Comment
01546BCE  896C24 00       mov     [esp],ebp          // especie de PUSH EBP
01546BD2  8BEC            mov     ebp,esp
01546BD4  81EC 0C000000   sub     esp,0C
01546BDA  66:8105 E46B54  add     word ptr [1546BE4],73FE
01546BE3  EB 04           jmp     short 01546BE9
01546BE5  59              pop     ecx
01546BE6  208D 6424D5EB   and     [ebp+EBD52464],cl
01546BEC  02CD            add     cl,ch
```

Como apunto en el comentario en la primera línea se ejecuta una especie de PUSH EBP, aunque no es del todo real pues debería ser:

MOV [ESP+4], EBP

para ser un PUSH EBP en condiciones, lo que ocurre es que un poco antes empujó un valor a la pila que sobrescribe ahora por el de EBP, esto sumado a la instrucción comentada sí hace un PUSH EBP, después vienen otras dos instrucciones bastante fáciles de detectar, pero luego se complica pues hace algunos PUSH, POP, etc.. y no se distingue bien cuáles son stolen bytes y cuales no, tenemos varias alternativas, entre ellas está copiar este bloque de memoria donde se ejecutan los stolen bytes y cargarlo con VirtualAlloc ene. dumpeado y luego saltar allí para ejecutarlos, otra opción que también me gusta es la de buscarlos en la pila en el OEP, lo más interesante que vi fue:

```
015370A2    896C24 00        mov     [esp],ebp
015370A6    8BEC             mov     ebp,esp
015370A8    81EC 0C000000    sub     esp,0C
015370AE    66:8105 B870530>add      word ptr [15370B8],73FE
015370B7    EB 04            jmp     short 015370BD
015370CD    895C24 00        mov     [esp],ebx
015370D1    66:8105 DB70530>add      word ptr [15370DB],73FE
015370DA    EB 02            jmp     short 015370DE
015370F0    897424 00        mov     [esp],esi
```

```
015370F4    66:8105 FE70530>add     word ptr [15370FE],73FE
015370FD    EB 04           jmp     short 01537103
01537113    897C24 00       mov     [esp],edi                       ; user32.77D121AE
01537117    66:8105 2171530>add     word ptr [1537121],73FE
01537120    EB 04           jmp     short 01537126
0153719F    897C24 04       mov     [esp+4],edi                     ; user32.77D121AE
015371A3    3E:EB 01        jmp     short 015371A7                  ; Superfluous prefix
015371F8    894C24 04       mov     [esp+4],ecx
015371FC    3E:EB 02        jmp     short 01537201                  ; Superfluous prefix
01537346    52              push    edx
01537347    F3:             prefix rep:                             ; Superfluous prefix
01537348    EB 02           jmp     short 0153734C
0153734A    CD 20           int     20
0153734C    8F4424 00       pop     dword ptr [esp]
01537350    EB 01           jmp     short 01537353
0153735C    895C24 04       mov     [esp+4],ebx
01537360    F2:             prefix repne:                           ; Superfluous prefix
01537361    EB 01           jmp     short 01537364
```

Si miro la pila Después de su ejecución veo:

```
0012FFA8   77D121AE  user32.77D121AE        \\
0012FFAC   0012CD34                         ||
0012FFB0   7FFDF000                         ||
0012FFB4   7FFDF000                         ||- Stolen Bytes!!
0012FFB8   7FFE0304                         ||
0012FFBC   0153705F                         ||
0012FFC0   0012FFF0                         //
0012FFC4   77E614C7  RETURN to kernel32.77E614C7
0012FFC8   77D121AE  user32.77D121AE
0012FFCC   0012CD34
0012FFD0   7FFDF000
```

Empujó exactamente 7 DWORDS, si miramos el valor de los registros [en el dumpeado] vemos que los stolen bytes quedarán así:

```
Push ebp
Mov ebp, esp
Add esp, -0C
Push edx
Push ebx
Push ebx
Push esi
Push edi
Mov eax, 5f6998
```

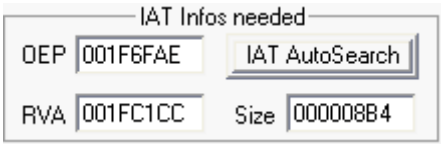
Bytes: 55 8B EC 83 C4 FC 52 53 53 56 57 B8 98 69 5F 00

Los pegamos antes del falso OEP:

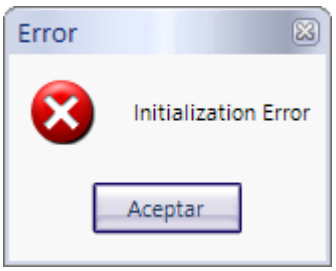
```
Address   Hex dump      Disassembly                    Comment
005F6FAE  55            push    ebp                    // OEP==5F6FAE
005F6FAF  8BEC          mov     ebp,esp
005F6FB1  83C4 FC       add     esp,-4
005F6FB4  52            push    edx
005F6FB5  53            push    ebx
005F6FB6  53            push    ebx
005F6FB7  56            push    esi
005F6FB8  57            push    edi
005F6FB9  B8 98695F00   mov     eax,005F6998
005F6FBE  E8 DD0FE1FF   call    00407FA0
005F6FC3  68 98705F00   push    005F7098               ASCII "TAG_AND_RENAME"
005F6FC8  6A FF         push    -1
005F6FCA  6A 00         push    0
005F6FCC  E8 CB10E1FF   call    0040809C
005F6FD1  A3 74BF5F00   mov     [5FBF74],eax
```

Nos queda el OEP en :005F6FAE, este valor lo utilizaremos ahora en el ImpREC, de momento dumpeamos con la casilla REBUILD IMPORT destildada puesto que repararemos la tabla nosotros y no queremos que Olly lo haga [no lo hace bien]. Dejamos el dumpeado en la carpeta de instalación del programa con el nombre tute.exe por decir algo.

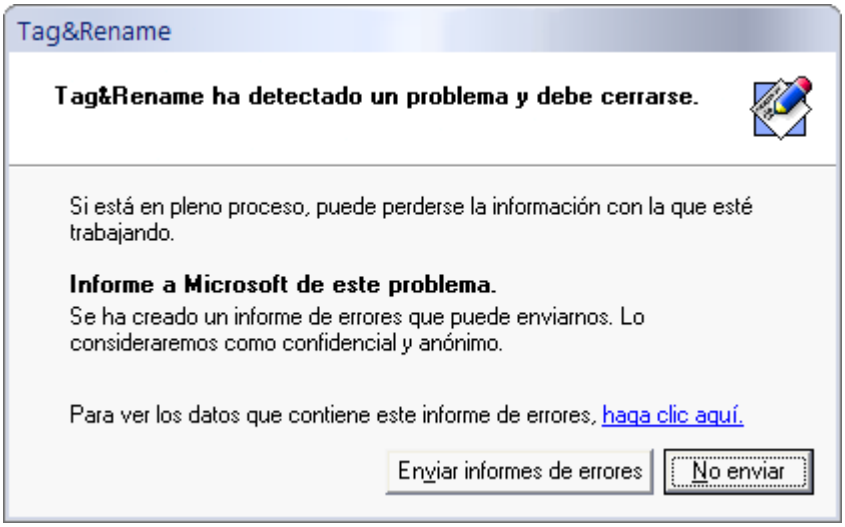
Abrimos el ImpREC si lo habíamos cerrado y seleccionamos el proceso TagRename de la lista ATTACH TO AN ACTIVE PROCESS y pulsamos LOAD TREE, le damos IAT.TXT que es el que salvamos antes y modificamos el valor del OEP, debe quedar así:



Pulsamos FIX DUMP y reparamos tute.exe, nos dejará tute_.exe al lado con la tabla reparada, pero si lo ejecutamos no funciona ¿? sale este error:



Lo acepto y sale otro [Este del sistema]:



Le digo NO EVIAR y cargo el dumpeado en el Olly, por si acaso le cambio el nombre por el original "TagRename.exe" pero tampoco corre. El error ocurre en esta llamada [CALL EAX]:

Address	Hex dump	Disassembly		Comment
005F7000	. 64:8920	mov	fs:[eax],esp	
005F7003	. 68 CC804000	push	<jmp.&kernel32.FatalExit>	
005F7008	. B8 04A75F00	mov	eax,005FA704	
005F700D	. FFD0	call	eax	tute_.005FA704
005F700F	. A1 A8AA5F00	mov	eax,[5FAAA8]	
005F7014	. 8B00	mov	eax,[eax]	
005F7016	. E8 2D63E6FF	call	0045D348	
005F701B	. A1 A8AA5F00	mov	eax,[5FAAA8]	

Dentro del call falla aquí:

Address	Hex dump	Disassembly		Comment
005FA706	8B40 02	mov	eax,[eax+2]	
005FA709	8B00	mov	eax,[eax]	
005FA70B	FF30	push	dword ptr [eax]	
005FA70D	8F00	pop	dword ptr [eax]	// Problemas..
005FA70F	FFE3	jmp	ebx	
005FA711	8D40 00	lea	eax,[eax]	
005FA714	E8 90400050	call	505FE7A9	
005FA719	90	nop		

Si miro en la ventana de registros veo EAX==77E93F25 dirección de FatalExit dentro de la librería kernel32, al hacer POP trata de escribir allí y por eso falla, una solución sería NOPear el POP y el PUSH anterior para no desajustar la pila, pero hay un problemilla más, si miro la zona en el OEP veo esto:

Address	Hex dump	Disassembly		Comment
005FA706	8D42 04	lea	eax,[edx+4]	
005FA709	8D02	lea	eax,[edx]	
005FA70B	0132	add	[edx],esi	
005FA70D	91	xchg	eax,ecx	// Problemas..
005FA70E	0201	add	al,[ecx]	
005FA710	E5 8D	in	eax,8D	I/O command
005FA712	40	inc	eax	
005FA713	00E8	add	al,ch	
005FA715	90	nop		

Está encriptada así que antes de parchearla debo encontrar el lugar de desencriptación de ésta, reinicio Olly con CTRL+F2 y veo que lo hace aquí:

Address	Hex dump	Disassembly		Comment
005F6FE4	BA 0D000000	mov	edx,0D	
005F6FE9	B8 04A75F00	mov	eax,005FA704	
005F6FEE	8028 02	sub	byte ptr [eax],2	// InicioLoop
005F6FF1	40	inc	eax	
005F6FF2	4A	dec	edx	
005F6FF3	75 F9	jnz	short 005F6FEE	// FinLoop
005F6FF5	33C0	xor	eax,eax	
005F6FF7	55	push	ebp	

Entonces deberé NOPear este bucle también al igual que el PUSH y el POP y después salvar los cambios al ejecutable con COPY TO EXECUTABLE FILE - ALL MODIFICATIONS y después SAVE FILE, y con esto ya está superado el primer escollo, como al final del bucle EDX queda a cero y antes se hace EDX==0Dh lo mejor será NOPear también esta instrucción, al final queda así:

Address	Hex dump	Disassembly		Comment
005F6FDE	64:FF31	push	dword ptr fs:[ecx]	
005F6FE1	64:8921	mov	fs:[ecx],esp	
005F6FE4	90	nop		
005F6FE5	90	nop		
005F6FE6	90	nop		
005F6FE7	90	nop		
005F6FE8	90	nop		
005F6FE9	90	nop		
005F6FEA	90	nop		
005F6FEB	90	nop		
005F6FEC	90	nop		
005F6FED	90	nop		
005F6FEE	90	nop		// InicioLoop
005F6FEF	90	nop		
005F6FF0	90	nop		
005F6FF1	90	nop		
005F6FF2	90	nop		
005F6FF3	90	nop		// FinLoop
005F6FF4	90	nop		
005F6FF5	33C0	xor	eax,eax	

Además del PUSH y el POP:

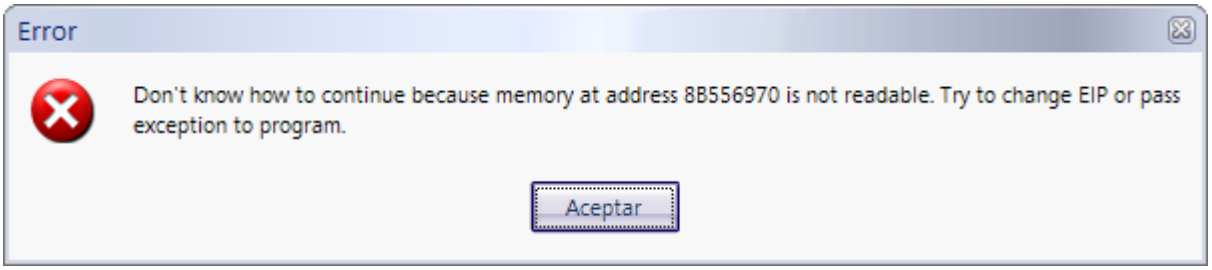
Address	Hex dump	Disassembly		Comment
005FA706	8B40 02	mov	eax,[eax+2]	
005FA709	8B00	mov	eax,[eax]	
005FA70B	90	nop		
005FA70C	90	nop		
005FA70D	90	nop		// Problemas..
005FA70E	90	nop		
005FA70F	FFE3	jmp	ebx	

Como digo salvamos los cambios al EXE. No sé porqué pero no me deja hacer el cambio en ambos lados a la misma vez, entonces salvo una parte, abro de nuevo el ejecutable en Olly y salvo la otra. Pruebo a ejecutar el ejecutable modificado y funciona a la primera, jeje..

Ahora que parece que el packer está K.O. procederemos a dejar el programa totalmente funcional, pues sale una nagscreen que nos informa del tiempo que nos queda dentro de los 30 días permitidos del período Trial.

Si surgieran imprevistos con algún antidump o algo por el estilo lo trataremos sobre la marcha tal y como vayan apareciendo, pero esto es una suposición, en principio no hay nada que nos deba molestar.

Pues vaya! si antes lo digo, al cerrarlo aparece el mismo mensaje de antes, pero esta vez viene acompañado de este otro:



Lo que ocurre es que al colocar los stolen bytes chafamos un valor necesario para el programa, el error ocurre en este loop:

```
Address   Hex dump   Disassembly                          Comment
0040425B  7E 27      jle     short 00404284
0040425D  FF4D FC    dec     dword ptr [ebp-4]            // inicio del loop
00404260  8B45 FC    mov     eax,[ebp-4]
00404263  A3 B0B45F00 mov    [5FB4B0],eax
00404268  8B45 F8    mov     eax,[ebp-8]
0040426B  8B55 FC    mov     edx,[ebp-4]
0040426E  8B44D0 04  mov     eax,[eax+edx*8+4]            // lee el valor
00404272  8945 F4    mov     [ebp-C],eax
00404275  837D F4 00 cmp     dword ptr [ebp-C],0
00404279  74 03      je      short 0040427E
0040427B  FF55 F4    call    [ebp-C]                      // salta a la funcion..
0040427E  837D FC 00 cmp     dword ptr [ebp-4],0
00404282  7F D9      jg      short 0040425D               // final del loop
00404284  33C0       xor     eax,eax
```

Estamos ante la tabla de inicialización pero en modo inversa, es decir, cuando ejecutamos el programa la citada tabla, y al finalizarlo llama una muy parecida que en lugar de inicializar lo que hace es desinicializar, en realidad sería la tabla de desinicialización [deinit table] creo que se llama, y el problema concretamente está en que en lugar de leer una dirección correcta no lo hace debido a los stolen bytes que chafan dicho valor:

```
Address   Hex dump   Disassembly                          Comment
005F6FA7  0000       add     [eax],al
005F6FA9  0000       add     [eax],al
005F6FAB  0070 69    add     [eax+69],dh
005F6FAE  55         push    ebp                          // OEP==5F6FAE
005F6FAF  8BEC       mov     ebp,esp
005F6FB1  83C4 FC    add     esp,-4
005F6FB4  52         push    edx
005F6FB5  53         push    ebx
005F6FB6  53         push    ebx
005F6FB7  56         push    esi
005F6FB8  57         push    edi
005F6FB9  B8 98695F00 mov    eax,005F6998
005F6FBE  E8 DD0FE1FF call   00407FA0
005F6FC3  68 98705F00 push   005F7098                     ASCII "TAG_AND_RENAME"
```

Esos bytes reemplazan a los originales entre los que están los que busca, si llegamos al OEP del empakado vemos cuáles son:

```
Address   Hex dump                                         ASCII
005F6FAC  70 69 5F 00 00 00 00 00 00 00 00 00 00 00 00 00  pi_.............
005F6FBC  00 00 E8 DD 0F E1 FF 68 98 70 5F 00 6A FF 6A 00  ..èÝ¤áÿh˜p_.jÿj.
005F6FCC  E8 CB 10 E1 FF A3 74 BF 5F 00 33 C9 55 68 86 70  èÊ+áÿft¿_.3ÉUh†p
005F6FDC  5F 00 64 FF 31 64 89 21 BA 0D 00 00 00 B8 04 A7  _.dÿ1d‰!º.....¸§
005F6FEC  5F 00 80 28 02 40 4A 75 F9 33 C0 55 68 47 70 5F  _.€(.@Juù3ÀUhGp_
005F6FFC  00 64 FF 30 64 89 20 68 CC 80 40 00 B8 04 A7 5F  .dÿ0d‰ hÌ€@.¸§_
```

Se trata de 0x5F6970 y la solución sería dejar estos bytes en su sitio y ejecutar los stolen bytes en otra parte, veamos cuantos son:

```
Address   Hex dump                                         ASCII
005F6FAC  70 69 55 8B EC 83 C4 FC 52 53 53 56 57 B8 98 69  piU‹ì‹Ä‹RSSVW¸˜i
005F6FBC  5F 00 E8 DD 0F E1 FF 68 98 70 5F 00 6A FF 6A 00  _.èÝ¤áÿh˜p_.jÿj.
005F6FCC  E8 CB 10 E1 FF A3 74 BF 5F 00 33 C9 55 68 86 70  èÊ+áÿft¿_.3ÉUh†p
005F6FDC  5F 00 64 FF 31 64 89 21 90 90 90 90 90 90 90 90  _.dÿ1d‰!□□□□□□□□
005F6FEC  90 90 90 90 90 90 90 90 90 33 C0 55 68 47 70 5F  □□□□□□□□□3ÀUhGp_
```

Vemos que el primer DWORD es 0x8B556970 en lugar del 0x005F6970 que debería haber, por esto falla el dumpeado, aquí copiaremos los bytes originales y colocaremos un JMP que nos llevará a una segunda zona donde se ejecutarán los stolen bytes, nada complicado.

Los bytes originales los obtengo del empakado, los pego en el dump y salvo los cambios, lo vuelvo a abrir con Olly y antes del call inserto un JMP a una zona libre donde colocaré los stolen:

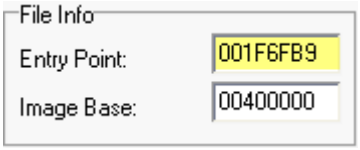
```
Address   Hex dump     Disassembly                          Comment
005F6FA9  0000         add     [eax],al
005F6FAB  0070 69      add     [eax+69],dh      <---- bytes restaurados!
005F6FAE  5F           pop     edi
005F6FAF  0000         add     [eax],al
005F6FB1  0000         add     [eax],al
005F6FB3  0000         add     [eax],al
005F6FB5  0000         add     [eax],al
005F6FB7  0000         add     [eax],al
005F6FB9  E9 26010000  jmp     005F70E4         <---- salto al injerto!
005F6FBE  E8 DD0FE1FF  call    00407FA0
005F6FC3  68 98705F00  push    005F7098                     ASCII "TAG_AND_RENAME"
```

Así queda al final el OEP que ahora estará en :5F6FB9, este valor lo podemos cambiar con PEditor v1.7 que va muy bien, aconsejo tenerlo modificado como indicó Arapumk en uno de sus fantásticos tutes para que no inserte basura en la cabecera por si tenemos que deshacer cambios, siempre es mejor que quede limpia y este programa como la mayoría de editores inserta una cadena.

Sólo nos queda ir a esa zona e injertar los stolen bytes + un salto de vuelta al código principal del programa, quedará algo así:

```
Address   Hex dump     Disassembly                          Comment
005F70E4  55           push    ebp
005F70E5  8BEC         mov     ebp,esp
005F70E7  83C4 FC      add     esp,-4
005F70EA  52           push    edx
005F70EB  53           push    ebx
005F70EC  53           push    ebx
005F70ED  56           push    esi
005F70EE  57           push    edi
005F70EF  B8 98695F00  mov     eax,005F6998
005F70F4  68 BE6F5F00  push    005F6FBE
005F70F9  C3           ret
005F70FA  0000         add     [eax],al
005F70FC  0000         add     [eax],al
```

Salvamos los cambios una vez más y modificamos el OEP antes que se nos olvide:



Picamos en APPLY CHANGES y ya tenemos el dumpeado listo!!

Lo siguiente que haremos será ir a por las limitaciones que pueda tener, de inicio nos muestra una nag informando de los días restantes del período Trial [30 días] y no sé si habrá algo más. Vamos a por él XDD

Eliminando el TimeTrial..

En realidad no nos debemos preocupar demasiado porque era el propio packer el que se encargaba del conteo de días, al ejecutar el dumpeado no habrá ningún tipo de chequeo y entramos bien al programa, aún habiendo pasado más de 30 días. De todos modos seguimos viendo una nag y cadenas tipo UNREGISTERED en el About, barra de título, etc..,

De momento cargamos el dumpeado en Olly, analizo el módulo con CTRL+A y miro entre las cadenas [strings] del programa, y veo esto:

```
Address   Hex dump     Disassembly                          Comment
005F6706  .  E8 0122E7FF  call    0046890C                  // hmmm..
005F670B  .  84C0         test    al,al
005F670D  .v 75 74        jnz     short 005F6783
005F670F  .  A1 68A85F00  mov     eax,[5FA868]
005F6714  .  8B00         mov     eax,[eax]
005F6716  .  BA F0675F00  mov     edx,005F67F0              ASCII "Tag&Rename 2.1.7.4 UNREGISTERED"
005F671B  .  E8 7810E4FF  call    00437798
005F6720  .  8B0D 08A85F00 mov    ecx,[5FA808]              tute_.005FBE1C
005F6726  .  A1 A8AA5F00  mov     eax,[5FAAA8]
```

No parece que vaya a ser muy complicado a partir de aquí, jeje.. ese call tiene toda la pinta de verificar algo.. ¿el serial?

A grandes rasgos lo que hace en el call es verificar el serial que hayamos introducido desde el menú HELP - REGISTER, y que guarda aquí:

```
[HKEY_CURRENT_USER\Software\Softpointer\Tag&Rename\Config]
"cbVQFFtoTagReplaseUnde"="12121212"
"Name"="SyXe'05"
```

Esos son los valores que yo puse, ya sabemos donde los guarda, pero el algoritmo no es muy alentador pues es del tipo HASH, al final la clave está aquí:

```
Address   Hex dump     Disassembly                          Comment
004689F8  > 8D55 DC    lea     edx,[ebp-24]
004689FB  . 8D45 C8    lea     eax,[ebp-38]
004689FE  . E8 4DFDFFFF call    00468750
00468A03  . 84C0       test    al,al
00468A05  .v 74 06     je      short 00468A0D
00468A07  . C645 FF 01 mov     byte ptr [ebp-1],1            // flag de exito ;)
00468A0B  .v EB 15     jmp     short 00468A22
00468A0D  > 8D55 C8    lea     edx,[ebp-38]
00468A10  . B9 14000000 mov    ecx,14
00468A15  . 8B45 F8    mov     eax,[ebp-8]
00468A18  . 8B18       mov     ebx,[eax]
00468A1A  . FF53 04    call    [ebx+4]
00468A1D  . 83F8 14    cmp     eax,14
00468A20  .^ 74 D6     je      short 004689F8
00468A22  > 33C0       xor     eax,eax
```

Hay que llegar a la dirección :00468A07 donde se modifica [ebp-1] pero más interesante sería parchear el call previo para que retorne AL=!0, donde hay:

```
Address   Hex dump     Disassembly                          Comment
0046876E  . 8B0424     mov     eax,[esp]
00468771  . 3B4424 14  cmp     eax,[esp+14]
00468775  .v 75 28     jnz     short 0046879F
00468777  . 8B4424 04  mov     eax,[esp+4]
0046877B  . 3B4424 18  cmp     eax,[esp+18]
0046877F  .v 75 1E     jnz     short 0046879F
00468781  . 8B4424 08  mov     eax,[esp+8]
00468785  . 3B4424 1C  cmp     eax,[esp+1C]
00468789  .v 75 14     jnz     short 0046879F
0046878B  . 8B4424 0C  mov     eax,[esp+C]
0046878F  . 3B4424 20  cmp     eax,[esp+20]
00468793  .v 75 0A     jnz     short 0046879F
00468795  . 8B4424 10  mov     eax,[esp+10]
00468799  . 3B4424 24  cmp     eax,[esp+24]
0046879D  .v 74 04     je      short 004687A3
0046879F  > 33C0       xor     eax,eax                       // flag de error ;(
004687A1  .v EB 02     jmp     short 004687A5
004687A3  > B0 01      mov     al,1                          // flag de exito ;)
004687A5  > 83C4 28    add     esp,28
004687A8  . 5F         pop     edi
004687A9  . 5E         pop     esi
004687AA  . C3         ret
```

Son cuatro DWORDS resultantes del HASH obtenido del serial que se verifican comparándose con los correctos, lo mejor sería NOPear los cuatro primeros saltos y JMPear el quinto hacia la zona buena:

```
Address   Hex dump     Disassembly                          Comment
0046876E  . 8B0424     mov     eax,[esp]
00468771  . 3B4424 14  cmp     eax,[esp+14]
00468775  . 90         nop
00468776  . 90         nop
00468777  . 8B4424 04  mov     eax,[esp+4]
0046877B  . 3B4424 18  cmp     eax,[esp+18]
0046877F  . 90         nop
00468780  . 90         nop
00468781  . 8B4424 08  mov     eax,[esp+8]
00468785  . 3B4424 1C  cmp     eax,[esp+1C]
00468789  . 90         nop
0046878A  . 90         nop
0046878B  . 8B4424 0C  mov     eax,[esp+C]
0046878F  . 3B4424 20  cmp     eax,[esp+20]
00468793  . 90         nop
00468794  . 90         nop
00468795  . 8B4424 10  mov     eax,[esp+10]
00468799  . 3B4424 24  cmp     eax,[esp+24]
0046879D  .v EB 04     jmp     short 004687A3
0046879F  . 33C0       xor     eax,eax                       // flag de error ;(
004687A1  .v EB 02     jmp     short 004687A5
004687A3  > B0 01      mov     al,1                          // flag de exito ;)
004687A5  > 83C4 28    add     esp,28
004687A8  . 5F         pop     edi
004687A9  . 5E         pop     esi
004687AA  . C3         ret
```

Seleccionamos la zona modificada, picamos con el botón derecho y elegimos la opción COPY TO EXECUTABLE - SELECTION y después SAVE FILE, podemos renombrar el ejecutable como tute_1.exe por si algo sale mal, si lo probamos funka perfecto, jeje.. si miramos en el About veremos nuestro nombre:

Ya podemos adelantar/retrasar el reloj, etc.. que seguirá funcionando, también han desaparecido todas las referencias a cadenas tipo UNREGISTERED, TRIAL y cosas así por lo que se puede decir que está crackeado xDD

Y hasta aquí el tute de hoy, algo sencillito para no complicarnos demasiado la vida, espero volver pronto con más trabajos, y como se suele decir en mi tierra mañana más y mejor!!

Un saludo a TODOS los CracksLatinoS sin EXCEPCIÓN XDD. Para cualquier cosa ya sabéis donde contactarme:

syxe05@gmail.com